

0.1 Hợp đồng thông minh

0.1.1 Công nghệ sử dụng

Các hợp đồng thông minh của hệ thống Lending Pool được phát triển trên nền tảng Ethereum, sử dụng các công nghệ và công cụ sau:

- Solidity ^0.8.28: Ngôn ngữ lập trình bậc cao dùng để xây dựng smart contract chạy trên Ethereum Virtual Machine (EVM). Phiên bản 0.8.28 bao gồm nhiều cải tiến về bảo mật so với các phiên bản cũ hơn, đặc biệt là kiểm tra overflow mặc định và cú pháp custom error tiết kiệm gas [1].
- Hardhat ^2.26.5: Framework phát triển smart contract chạy trên Node.js, hỗ trợ biên dịch, kiểm thử, deploy, verify smart contract, cũng như mô phỏng và fork blockchain phục vụ quá trình phát triển [2].
- OpenZeppelin Contracts ^5.4.0 và Contracts Upgradeable ^5.6.1: Bộ thư viện smart contract chuẩn đã qua kiểm định bảo mật, được sử dụng cho các tính năng: UUPS upgradeable proxy, ReentrancyGuardTransient, SafeERC20, PausableUpgradeable, AccessControl, và TimelockController [3].
- Chainlink Contracts ^1.5.0: Interface AggregatorV3 để đọc Chainlink Data Feed [4].
- Hardhat Gas Reporter ^2.3.0: Đo lường chi phí gas của từng function trong test.
- TypeScript ^5.9.3 + Ethers.js ^6.15.0: Ngôn ngữ lập trình có kiểu tĩnh và thư viện tương tác blockchain, được dùng cho toàn bộ script deploy và test suite [5].
- Chai ^4.5.0 + Mocha: Assertion framework dùng cho kiểm thử tự động unit test.

0.1.2 Cấu trúc codebase

```
.
|-- contracts
|   |-- InterestRateModel.sol
|   |-- LendingPool.sol
|   |-- LendingPoolStorage.sol
|   |-- Liquidation.sol
|   |-- MyOracle.sol
|   |-- PriceRouter.sol
|   |-- PriceRouterStorage.sol
|   |-- ProtocolController.sol
|   |-- ProtocolTimelock.sol
|   |-- interfaces
```

```

|   |   `-- Interfaces.sol
|   `-- test
|   `-- MockERC20.sol
|-- deployments
|   |-- abis.json
|   `-- addresses.json
|-- hardhat.config.ts
|-- scripts
|   |-- deploy-protocol
|   |   |-- config.ts
|   |   |-- deploy.ts
|   |   |-- phases
|   |   |   |-- index.ts
|   |   |   |-- phase1.ts
|   |   |   |-- phase2.ts
|   |   |   `-- phase3.ts
|   |   `-- utils
|   |   |-- abis.ts
|   |   |-- index.ts
|   |   `-- save.ts
|   `-- deploy-safe.ts
|-- test
|   |-- Fixture.test.ts
|   |-- InterestRateModel.test.ts
|   |-- LendingPool.test.ts
|   |-- Liquidation.test.ts
|   `-- ProtocolController.test.ts
`-- tsconfig.json

```

0.1.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Xử lý số học fixed-point với uint256

Vấn đề: Solidity không hỗ trợ số thực (floating point). Phép tính lãi suất như $\text{borrowRate} \times \text{utilization} / \text{SECONDS_PER_YEAR}$ phải được thực hiện hoàn toàn bằng số nguyên mà không mất độ chính xác. Đây là nền tảng kỹ thuật của cơ chế tích lũy lãi Lazy Accrual được phân tích tại mục ??.

Giải pháp: Sử dụng $\text{SCALE} = 1e18$ như một hằng số biểu diễn số 1,0 trong không gian fixed-point. Mọi tỷ lệ phần trăm, lãi suất và hệ số đều được lưu dưới dạng số nguyên nhân với $1e18$. Quy tắc thực hiện phép tính: nhân trước, chia sau để tránh mất độ chính xác.

```
// Dung: nhan truoc, chia sau
result = (a * b) / SCALE

// Sai: chia truoc, nhan sau
result = (a / SCALE) * b
```

Ví dụ: lãi suất vay = 5% → lưu là $0.05 \times 10^{18} = 5 \times 10^{16}$. Khi tính $\text{interest} = \text{principal} \times \text{rate} / \text{SCALE}$, kết quả trả về đúng số tiền lãi tính bằng token gốc (với 18 decimals).

Đây là nền tảng kỹ thuật triển khai của mục ?? (Cơ chế tích lũy lãi Lazy Accrual).

b, Thiết kế storage gap tránh xung đột slot trong UUPS

Vấn đề: Khi upgrade implementation mới, nếu thêm biến state mới vào cuối smart contract LendingPool (không phải LendingPoolStorage), thứ tự slot bị dịch chuyển do cách Solidity kế thừa storage. Điều này dẫn đến đọc sai dữ liệu sau khi upgrade.

Giải pháp: Khai báo `__gap[39]` ở cuối LendingPoolStorage, dự trữ 39 slot cho các biến tương lai. Hiện tại LendingPoolStorage sử dụng 11 biến (11 slot), tổng cộng $11 + 39 = 50$ slot được dự trữ cho smart contract. PriceRouterStorage tương tự với `__gap[47]` (3 biến + 47 gap = 50 slot). Khi cần thêm biến mới, thu nhỏ `__gap[ ]` tương ứng — thứ tự các biến hiện tại không bị ảnh hưởng.

Con số 50 slot là convention được OpenZeppelin khuyến nghị trong tài liệu chính thức về Upgradeable Contracts, đủ cho hầu hết các trường hợp mở rộng trong vòng đời của một giao thức [3]. Khi cần thêm biến state mới trong một lần upgrade, chỉ cần khai báo biến mới ngay trước `__gap` và giảm kích thước `__gap` tương ứng — thứ tự các biến hiện tại không bị thay đổi và không có xung đột slot nào xảy ra.

Tham chiếu lý thuyết: Xem mục ?? về Storage Layout và rủi ro xung đột slot trong UUPS.

0.1.4 Quá trình triển khai

Sau khi hoàn tất cài đặt và kiểm thử, các smart contract được triển khai lên mạng Sepolia (Ethereum Testnet) theo quy trình 3 giai đoạn được tổ chức thành deploy script TypeScript riêng biệt, không sử dụng Hardhat Ignition. Lý do lựa chọn custom deploy script thay vì Ignition là vì Hardhat Ignition không hỗ trợ `@openzeppelin/hardhat-upgrades` — plugin cần thiết để deploy UUPS proxy và kiểm tra storage layout tự động khi upgrade. Toàn bộ script được điều phối bởi

deploy.ts, nhận thông tin cấu hình từ config.ts theo từng mạng (chainId), sau đó thực thi tuần tự 3 phase:

Phase 1 — Deploy: Triển khai tất cả smart contract lên blockchain theo thứ tự phụ thuộc:

1. Deploy 2 instance InterestRateModel: một cho stablecoin (USDC) và một cho tài sản biến động (WETH, WBTC) với tham số baseRate, slope1, slope2, optimalUtilization, reserveFactor
2. Deploy MyOracle — oracle tự quản lý để có thể tự đặt giá trong môi trường testnet.
3. Deploy PriceRouter dưới dạng UUPS proxy bằng upgrades.deployProxy() — plugin tự động deploy cả proxy contract lẫn implementation contract, kiểm tra storage layout, và lưu lại địa chỉ implementation để phục vụ upgrade sau này. Địa chỉ implementation được lấy qua upgrades.erc1967.getImplementationAddress().
4. Deploy LendingPool dưới dạng UUPS proxy tương tự, nhận địa chỉ PriceRouter và collateralFactor ngay trong lúc initialize().
5. Deploy Liquidation là smart contract thông thường (non-upgradeable), nhận địa chỉ PriceRouter, LendingPool và 3 tham số rủi ro: liquidationThreshold, closeFactor, liquidationIncentive.
6. Deploy ProtocolTimelock với cấu hình: proposer là địa chỉ Safe Multisig (đọc từ biến môi trường SAFE_ADDRESS), executor là address(0) (open execution — bất kỳ ai cũng có thể execute sau khi hết minDelay), và minDelay đọc từ TIMELOCK_MIN_DELAY_SECONDS [6].
7. Deploy ProtocolController nhận địa chỉ của tất cả smart contract vừa deploy và địa chỉ Timelock làm admin.

Sau khi deploy xong, địa chỉ mỗi smart contract (bao gồm cả proxy và implementation riêng biệt đối với LendingPool và PriceRouter) được lưu vào file deployments/addresses.json

Phase 2 — Wire: Kết nối các smart contract với nhau sau khi toàn bộ đã có địa chỉ. Trong phase này, LendingPool.setLiquidation() được gọi để LendingPool biết địa chỉ Liquidation — cần thiết để onlyLiquidation modifier hoạt động đúng.

Phase 3 — Migrate: Chuyển giao quyền kiểm soát từ deployer sang ProtocolController. Lần lượt gọi setController(controllerAddress) trên 4 smart contract: MyOracle, PriceRouter, LendingPool và Liquidation. Sau bước này, deployer không còn quyền admin trực tiếp — mọi hành động quản trị đều phải đi qua ProtocolCon-

troller và ProtocolTimelock (tức là qua Safe Multisig với minDelay). Đây là bước quan trọng nhất về mặt bảo mật, hoàn tất việc "chuyển giao quyền lực" cho cơ chế quản trị phi tập trung.

Sau khi 3 phase hoàn tất, exportABIs() tự động đọc artifact Hardhat đã biên dịch (từ thư mục artifacts/contracts/) và trích xuất ABI của tất cả 7 smart contract (LendingPool, PriceRouter, MyOracle, Liquidation, ProtocolTimelock, ProtocolController, InterestRateModel), lưu vào deployments/abis.json. File này được chia sẻ trực tiếp cho frontend và backend sử dụng khi khởi tạo contract instance.

0.2 Backend

0.2.1 Công nghệ sử dụng

Hệ thống backend được xây dựng trên nền tảng Node.js với TypeScript, sử dụng kiến trúc đa tiến trình event-driven. Các công nghệ và thư viện chính gồm:

- Node.js 22.19.0 + TypeScript ^5.9.3: Môi trường runtime JavaScript phía server với hệ thống kiểu tĩnh. TypeScript giúp phát hiện lỗi tại compile-time, đặc biệt quan trọng khi làm việc với BigInt (uint256 từ Solidity) và các kiểu dữ liệu phức tạp từ blockchain.
- Express.js ^5.1.0: Framework nhẹ để xây dựng RESTful API.
- Sequelize ^6.37.8 + PostgreSQL: ORM cho Node.js, hỗ trợ ánh xạ dữ liệu giữa PostgreSQL và các đối tượng TypeScript. PostgreSQL được chọn nhờ kiểu DECIMAL(78,0) có thể lưu chính xác số nguyên 256-bit từ Solidity.
- ethers.js ^6.15.0: Thư viện tương tác blockchain, dùng để kết nối với RPC node, đọc event, gọi hàm view trên smart contract và decode event log [5].
- RabbitMQ (amqplib ^0.10.9): Message broker trung gian để phối hợp giữa các tiến trình. Đảm bảo không mất event khi worker gặp sự cố [7].
- Redis (ioredis ^5.10.0): In-memory store phục vụ 2 mục đích: (1) làm trung gian pub/sub cho WebSocket, (2) Lưu trữ dữ liệu ngắn hạn (OTP, nonce).
- Zod ^4.3.6: Thư viện validate input và infer type tự động tích hợp với TypeScript.
- Socket.io ^4.8.1: Thư viện WebSocket, forward event từ Redis pub/sub đến frontend theo thời gian thực.
- Croner ^10.0.1: Thư viện cron job scheduler hỗ trợ timezone và chế độ protect.
- Nodemailer ^8.0.5: Thư viện gửi email qua SMTP.
- Jose ^6.2.3: Thư viện JWT hiện đại.

- Các thư viện khác: dayjs, dotenv,...

0.2.2 Cấu trúc codebase

Cấu trúc thư mục phân chia theo kiến trúc: src/apps/ chứa 5 entry point của 5 tiến trình độc lập, src/modules/ chứa business logic được tổ chức theo domain (blockchain, blockchain-worker, proposals, email, noti-worker), src/shared/ chứa các tiện ích dùng chung (database client, RabbitMQ helper, WebSocket publisher, config).

```
.
|-- infra
|   `-- docker-compose.local.yaml
|-- logs
|   |-- blc-indexer.log
|   |-- blc-worker.log
|   |-- croner.log
|   |-- http-server.log
|   |-- noti-worker.log
|   `-- seed.log
|-- src
|   |-- apps
|   |   |-- blc-indexer
|   |   |-- blc-worker
|   |   |-- croner
|   |   |-- http-server
|   |   |-- noti_proposal-worker
|   |   `-- seed
|   |-- models
|   |   |-- accrue_log.model.ts
|   |   |-- asset.model.ts
|   |   |-- asset_config.model.ts
|   |   |-- asset_snapshot.model.ts
|   |   |-- block.model.ts
|   |   |-- cronner_state.model.ts
|   |   |-- index.ts
|   |   |-- liquidatable_user.model.ts
|   |   |-- proposal.model.ts
|   |   |-- scanner.model.ts
|   |   |-- session.model.ts
|   |   |-- transaction.model.ts
|   |   |-- treasury_log.model.ts
|   |   |-- user.models.ts
```

```

|   |   |-- user_asset.model.ts
|   |   `-- user_snapshot.model.ts
|   |-- modules
|   |   |-- assets
|   |   |-- auth
|   |   |-- blockchain
|   |   |-- blockchain-worker
|   |   |-- email
|   |   |-- logs-chart
|   |   |-- multisig
|   |   |-- noti-worker
|   |   |-- proposals
|   |   |-- seed
|   |   |-- snapshots
|   |   |-- transactions
|   |   `-- users
|   `-- shared
|   |-- bootstrap
|   |-- config
|   |-- constants
|   |-- infra
|   |-- types
|   |-- utils
|   `-- ws
|-- tsconfig.json

```

0.2.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Lưu uint256 của Solidity vào PostgreSQL

Solidity sử dụng uint256 với giá trị tối đa khoảng 1.15×10^{77} , vượt xa giới hạn lưu trữ của hai tầng phía dưới: PostgreSQL BIGINT chỉ hỗ trợ tối đa khoảng 9.2×10^{18} , trong khi kiểu number của JavaScript chỉ biểu diễn chính xác số nguyên đến $2^{53} - 1$ ($\approx 9 \times 10^{15}$).

Để tránh mất độ chính xác, hệ thống sử dụng bigint native của JavaScript/TypeScript để xử lý toàn bộ giá trị uint256 ở tầng ứng dụng. Khi lưu xuống PostgreSQL, các giá trị này được chuyển sang dạng chuỗi thập phân và lưu bằng kiểu DECIMAL(78,0) (NUMERIC(78,0)), cho phép lưu chính xác số nguyên lên đến 78 chữ số.

Cách tiếp cận này đảm bảo dữ liệu blockchain được bảo toàn đầy đủ từ smart contract đến database mà không xảy ra overflow hoặc mất precision.

b, Đảm bảo idempotency khi xử lý event

Với at-least-once delivery của RabbitMQ, một event có thể được deliver nhiều lần nếu consumer crash sau khi xử lý nhưng trước khi ack — Phân tích chi tiết về idempotency và cách triển khai đã được trình bày tại mục **??**. Trong quá trình cài đặt, idempotency được áp dụng nhất quán:

- Bảng `accrue_logs`, `treasury_logs`, `transactions`: unique constraint trên (`transactionHash`, `logIndex`)
- Bảng `user_assets`: cột `lastSyncedBlock` và `lastSyncedLogIndex` ngăn event cũ ghi đè state mới
- Bảng `blocks`: unique constraint trên (`number`, `chainId`) ngăn duplicate block header

Tham chiếu: Mục **??**.

c, Xử lý SIWE và quản lý session

Vấn đề: backend cần xác thực người dùng mà không dùng username/password truyền thống. Đồng thời, cần hỗ trợ người dùng đăng nhập từ nhiều thiết bị cùng lúc và có khả năng thu hồi session cụ thể.

Giải pháp: Áp dụng EIP-4361 (Sign-In with Ethereum) — chuẩn được trình bày tại mục **??** [8]. Luồng cụ thể trong hệ thống:

- Frontend request nonce từ backend (GET `/api/auth/nonce?address=0x...`)
- Backend tạo nonce ngẫu nhiên, lưu vào Redis với TTL 5 phút
- Frontend tạo SIWE message chuẩn EIP-4361, yêu cầu ví ký
- Frontend gửi message + chữ ký lên backend (POST `/api/auth/verify`)
- Backend xác minh chữ ký bằng `ethers.verifyMessage()`, kiểm tra nonce khớp và chưa hết hạn
- Backend tạo access và refresh token (UUID), lưu refresh token vào bảng `sessions` với device/IP info, trả về access token
- Các request tiếp theo gửi kèm token trong Authorization header — backend lookup session trong database

Thiết kế nhiều session cho một địa chỉ: bảng `sessions` có thể chứa nhiều bản ghi cho cùng một `userAddress`, mỗi bản ghi là một thiết bị khác nhau. Người dùng có thể xem danh sách session đang hoạt động và thu hồi session cụ thể (set `revoked = true`).

Tham chiếu lý thuyết: Mục ?? — Sign-In with Ethereum (EIP-4361).

0.2.4 Quá trình triển khai

Hệ thống backend được triển khai dưới dạng 5 tiến trình Node.js độc lập. Mỗi tiến trình được khởi động bằng lệnh riêng từ package.json:

Tiến trình	Entry point	Vai trò
blc-indexer	apps/blc-indexer/index.ts	Quét blockchain, publish event vào RabbitMQ
blc-worker	apps/blc-worker/index.ts	Nhận message từ RabbitMQ, lưu database
croner	apps/croner/index.ts	Snapshot hằng ngày, quét Safe Multisig
noti-worker	apps/noti_proposal-worker/index.ts	Xử lý email OTP, proposal, thông báo admin
http-server	apps/http-server/index.ts	REST API và WebSocket

Bảng 0.1: Khởi động các tiến trình

Các bước triển khai:

Bước 1 — Cấu hình môi trường: Tạo file .env với các biến: RPC URL của từng chain, địa chỉ các smart contract đã deploy, DSN của PostgreSQL, URL của RabbitMQ, URL của Redis, thông tin SMTP cho email.

Bước 2 — Khởi động các tiến trình: Mỗi tiến trình được khởi động độc lập. Trong môi trường production có thể dùng PM2 hoặc Docker để quản lý vòng đời tiến trình và tự động restart khi gặp lỗi.

Bước 3 — Xác nhận hoạt động: Quan sát log của blc-indexer để xác nhận đang quét block và publish event đúng; quan sát log blc-worker để xác nhận đang nhận và xử lý message, quan sát log của các tiến trình khác để xác nhận hoạt động

0.3 Frontend

0.3.1 Công nghệ sử dụng

Phần giao diện người dùng được xây dựng bằng các công nghệ sau:

- React.js + Next.js (App Router): Framework React hỗ trợ điều hướng, Server-Side Rendering (SSR) và tối ưu hiệu năng.
- ethers.js v6: Tương tác với MetaMask và các smart contract trên blockchain [5].
- siwe (Sign-In with Ethereum): Thư viện triển khai chuẩn EIP-4361 [8].
- axios: Gửi HTTP request đến backend.
- socket.io-client: Lắng nghe sự kiện WebSocket từ backend theo thời gian thực.

- Material-UI (MUI): Thư viện UI component theo Material Design.

0.3.2 Cấu trúc codebase

```
|-- app
|   |-- admin
|   |   `-- page.tsx
|   |-- borrow
|   |   `-- page.tsx
|   |-- dashboard
|   |   `-- page.tsx
|   |-- history
|   |   `-- page.tsx
|   |-- layout.tsx
|   |-- liquidation
|   |   `-- page.tsx
|   |-- page.tsx
|   |-- proposals
|   |   `-- page.tsx
|   `-- supply
|   `-- page.tsx
|-- components
|   |-- BorrowDialog.tsx
|   |-- DepositDialog.tsx
|   |-- Navbar.tsx
|   |-- RepayDialog.tsx
|   |-- WalletConnect.tsx
|   |-- WithdrawDialog.tsx
|   `-- charts
|   `-- TimeSeriesChart.tsx
|-- hooks
|   `-- useContracts.ts
|-- jsconfig.json
|-- lib
|   |-- axios.ts
|   `-- web3.ts
|-- next-env.d.ts
|-- next.config.mjs
|-- services
|   |-- SafeMultisigService.ts
|   |-- assetService.ts
|   |-- authService.ts
|   |-- logService.ts
```

```

|   |-- marketConfigService.ts
|   |-- proposalService.ts
|   |-- snapshotService.ts
|   |-- transactionService.ts
|   |-- userAssetService.ts
|   `-- userService.ts
|-- tsconfig.json
|-- tsconfig.tsbuildinfo
`-- types
    |-- ethereum.d.ts
    `-- governance.ts

```

0.3.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Cách thức truy vấn dữ liệu: blockchain trực tiếp vs backend API

Frontend có hai nguồn dữ liệu: đọc trực tiếp từ blockchain qua ethers.js (dữ liệu real-time, luôn chính xác tuyệt đối), hoặc truy vấn từ backend API (nhanh hơn, có lịch sử).

Chiến lược phân chia được áp dụng:

- Đọc blockchain trực tiếp: số dư thực tế của người dùng (currentDeposit, currentBorrow), health factor trước khi ký transaction, preview kết quả của một giao dịch (previewDeposit, previewBorrow)
- Từ backend API: danh sách tài sản hỗ trợ và thông số, lịch sử giao dịch, biểu đồ TVL và lãi suất theo thời gian, danh sách vị thế liquidatable, danh sách proposals

Quyết định này tối ưu cả hai chiều: dữ liệu quyết định tài chính luôn lấy từ nguồn đáng tin cậy nhất (blockchain), trong khi dữ liệu hiển thị được lấy từ backend để giảm số lượng RPC call và hỗ trợ historical queries.

b, Hiển thị số dư lãi suất cập nhật mà không cần reload

Số dư lãi suất của người dùng tăng liên tục theo giây (do cơ chế Lazy Accrual của smart contract), nhưng database backend chỉ cập nhật khi có event mới. Nếu frontend chỉ lấy dữ liệu từ backend, số dư hiển thị sẽ luôn là số dư tại thời điểm event cuối cùng, không phản ánh lãi đã tích lũy từ đó đến nay. Giải pháp: frontend gọi trực tiếp hàm view trên blockchain (getPreviewUserDeposit, getPreviewUserBorrow) mỗi 15 giây — những hàm này tính toán số dư hiện tại có tính đến lãi tích lũy mà không tốn gas (vì là view function, không ghi blockchain).

c, Xử lý sự kiện thanh lý real-time qua WebSocket

Khi một vị thế rơi vào ngưỡng có thể bị thanh lý, thông tin cần được cập nhật ngay lập tức đến giao diện người dùng mà không cần polling. Frontend kết nối WebSocket với backend và lắng nghe sự kiện `LiquidatableUsersUpdated`. Mỗi khi `blc-indexer` hoàn tất quét vị thế (mỗi 30 giây), danh sách địa chỉ bị liquidatable được publish qua Redis → http-server nhận và forward qua WebSocket → frontend cập nhật UI. Thiết kế này đảm bảo liquidator có thể phát hiện cơ hội thanh lý sớm nhất có thể.

0.3.4 Quá trình triển khai

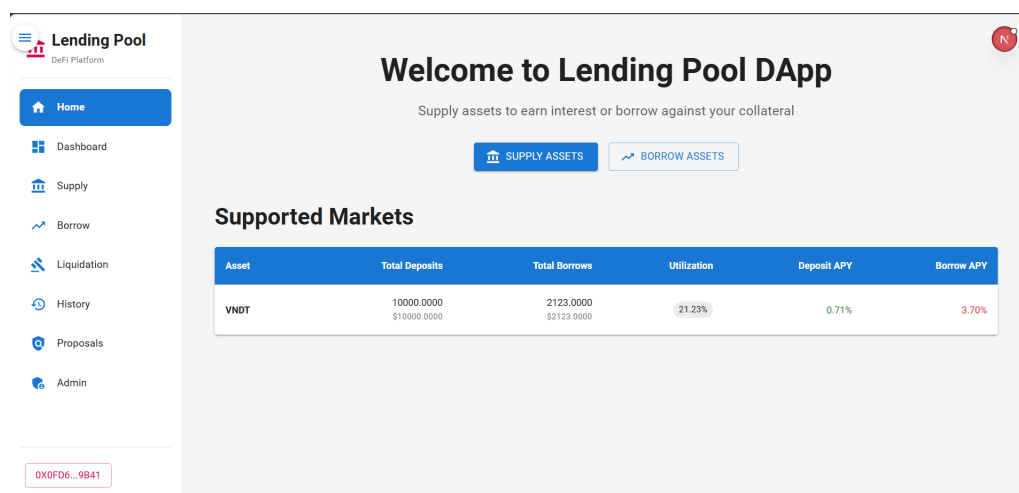
Bước 1 — Cấu hình biến môi trường: Đặt `NEXT_PUBLIC_BACKEND_URL` trỏ đến địa chỉ backend API, các biến `NEXT_PUBLIC_CONTRACT_` chứa địa chỉ từng smart contract trên Sepolia.

Bước 2 — Cấu hình ethers.js: Xây dựng thư viện tạo contract instance từ ABI và địa chỉ, bọc MetaMask bằng `BrowserProvider` để frontend có thể ký giao dịch.

Bước 3 — Triển khai production: Frontend được build và deploy lên Vercel (<https://lending-pool-prj3.vercel.app>). Vercel tự động nhận diện Next.js và cấu hình Server-Side Rendering. Mỗi lần push code lên GitHub, Vercel tự động trigger deploy lại.

0.3.5 Giao diện Demo

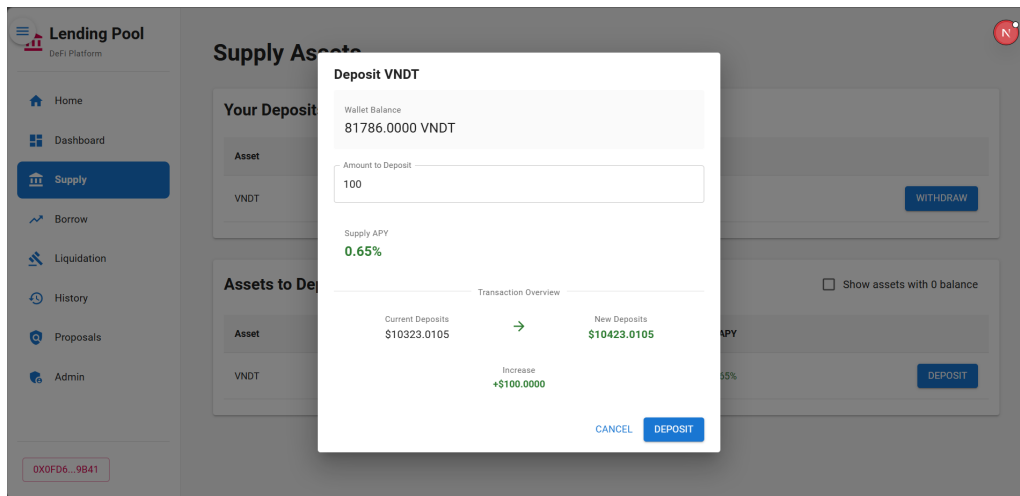
Hình 0.1 thể hiện trang chủ khi người dùng vừa vào ứng dụng, có danh sách và thông tin các tài sản đang được hỗ trợ trong giao thức.



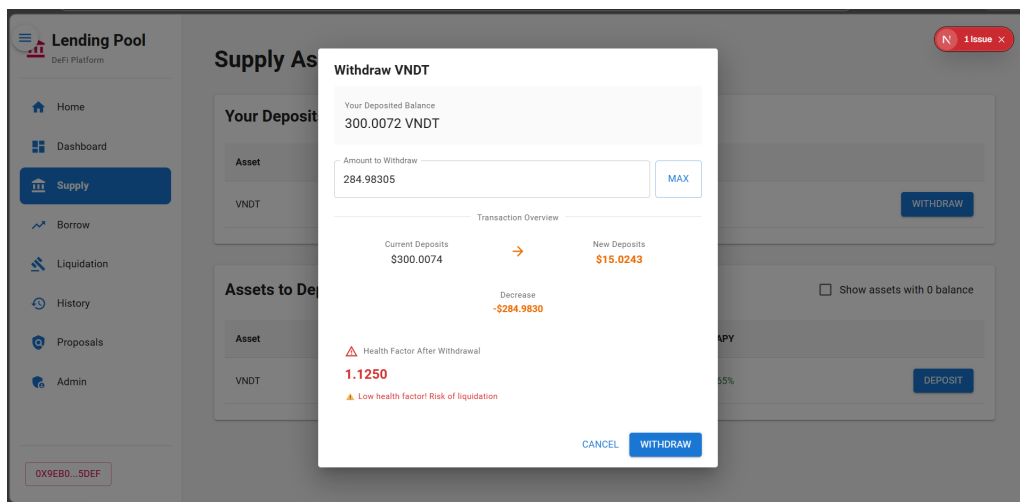
Hình 0.1: Trang chủ

Hình 0.2 biểu diễn trang gửi tài sản và form pop up cho biết số dư trong ví và trong giao thức của một tài sản, cùng với preview sau khi gửi tài sản, trong khi

hình 0.3 biểu diễn pop up form khi rút tài sản, cho biết số dư mới cùng với health factor sau khi rút tài sản.

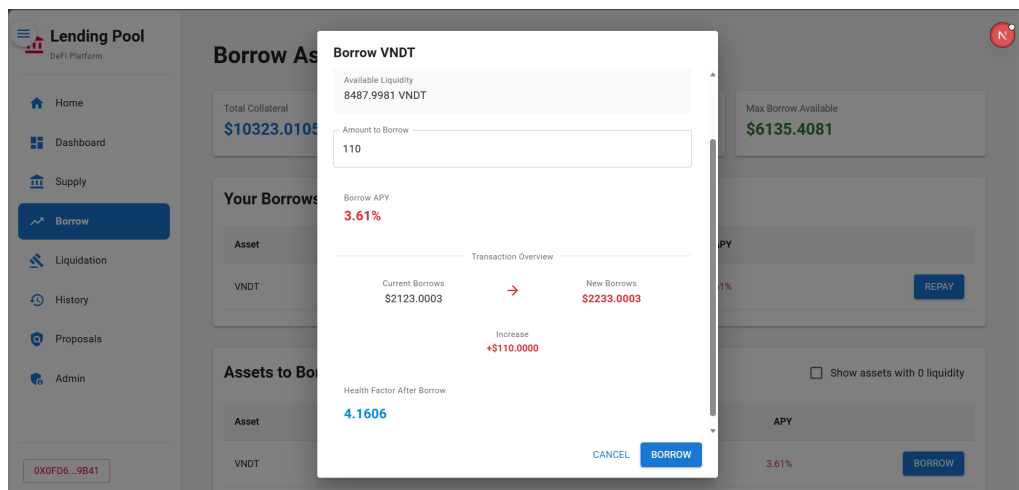


Hình 0.2: Pop up gửi tài sản

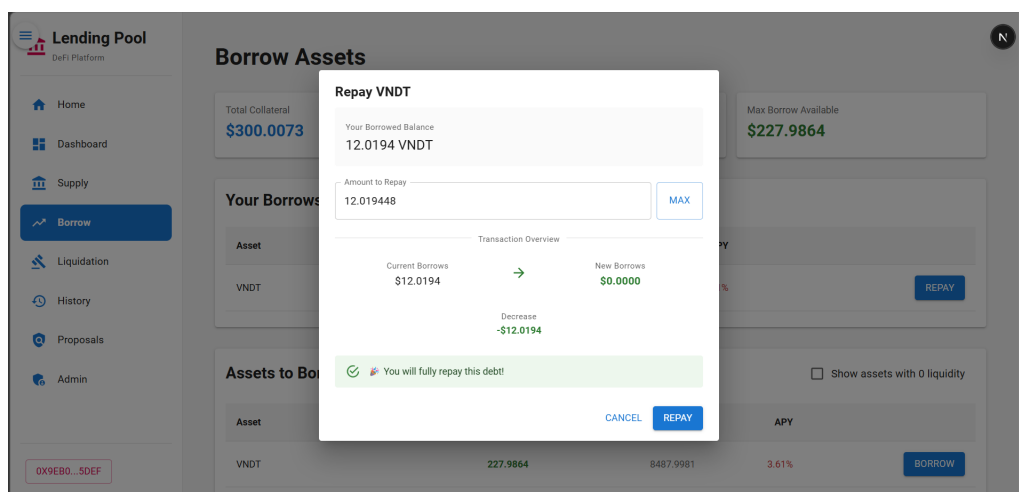


Hình 0.3: Pop up rút tài sản

Hình 0.4 biểu diễn trang vay tài sản và form pop up cho biết số tài sản có thể vay với thể chấp hiện tại, cùng với preview số dư và health factor sau khi vay tài sản, và hình 0.5 biểu diễn pop up form trả nợ tài sản cho biết số dư nợ của người dùng,

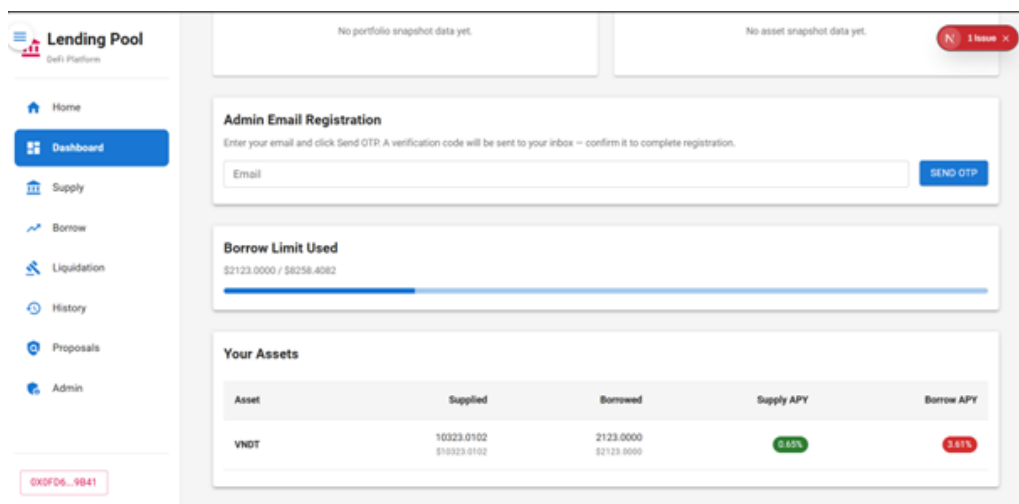


Hình 0.4: Pop up vay tài sản



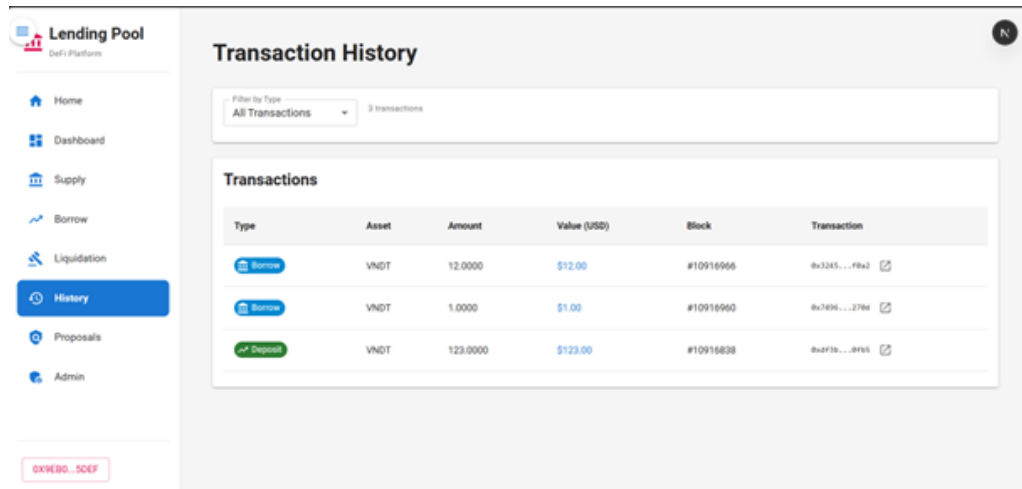
Hình 0.5: Pop up trả nợ tài sản

Hình 0.6 biểu diễn trang dashboard bao gồm các thông tin cá nhân của tài khoản ứng với địa chỉ xác định.



Hình 0.6: Trang dashboard cá nhân

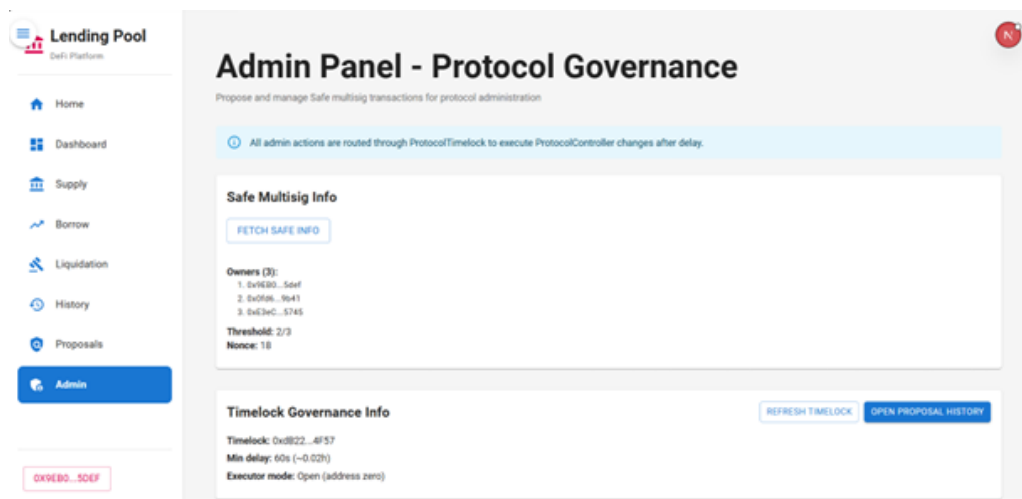
Hình 0.7 thể hiện lịch sử các giao dịch của người dùng tương tác với giao thức và các thông tin liên quan.



Type	Asset	Amount	Value (USD)	Block	Transaction
Borrow	VNDT	12.0000	\$12.00	#10916966	0x3245...f9a2
Borrow	VNDT	1.0000	\$1.00	#10916960	0x7496...2784
Deposit	VNDT	123.0000	\$123.00	#10916838	0x2f18...4f55

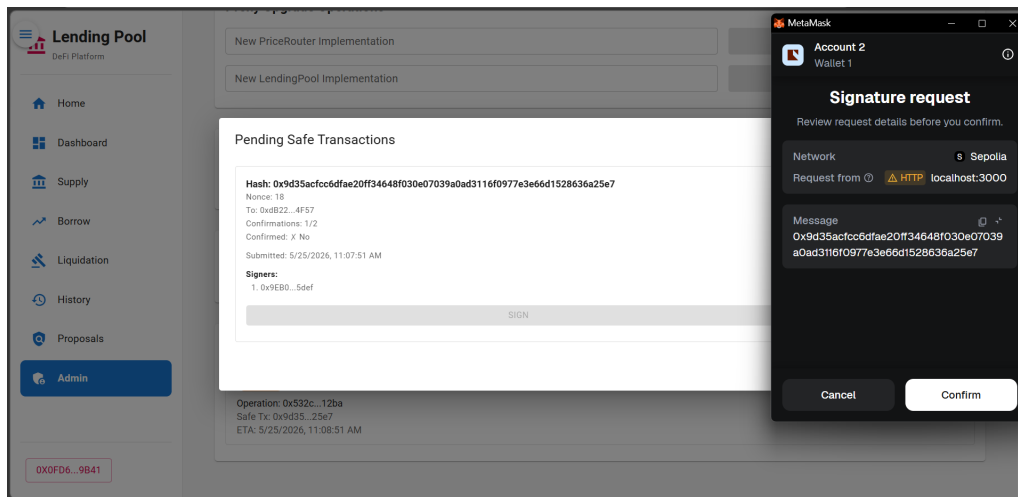
Hình 0.7: Trang lịch sử giao dịch cá nhân

Hình 0.8 biểu diễn trang admin dành cho quản trị hệ thống, chỉ có những admin được cấp quyền mới có thể truy cập trang này; hình 0.9 biểu diễn thao tác admin ký và thực thi đề xuất với ví đa chữ ký trong khi hình 0.10 thể hiện hàng đợi timelock của các đề xuất admin.

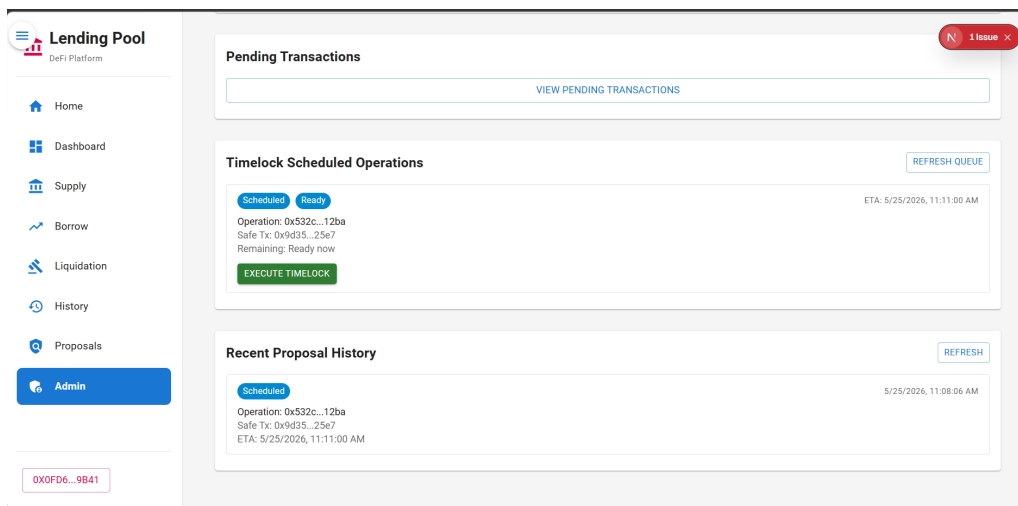


Admin Panel - Protocol Governance	
Propose and manage Safe multisig transactions for protocol administration	
All admin actions are routed through ProtocolTimelock to execute ProtocolController changes after delay.	
Safe Multisig Info	
Owners (3): 1. 0x1680...5def 2. 0x0f96...9a41 3. 0x03ac...5745	
Threshold: 2/3 Nonce: 18	
Timelock Governance Info	
Timelock: 0x0222...4f57 Min delay: 60s (~0.02h) Executor mode: Open (address zero)	

Hình 0.8: Trang admin panel

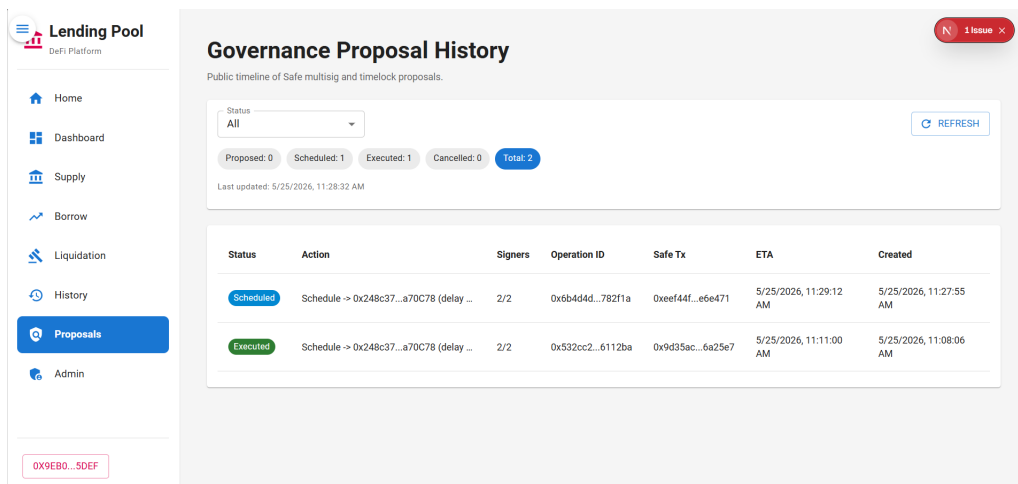


Hình 0.9: Admin tương tác với multisig



Hình 0.10: Hàng đợi timelock và danh sách đề nghị

Hình 0.11 trình bày danh sách các đề xuất đã từng xuất hiện của giao thức và các thông tin liên quan



Hình 0.11: Trang đề nghị thực thi